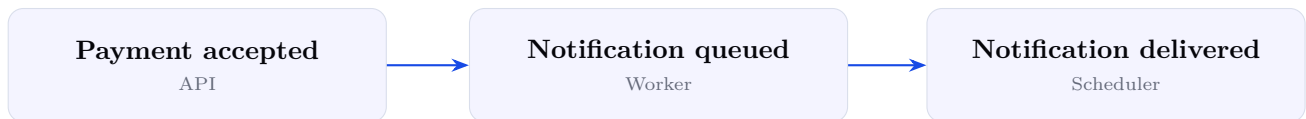


Everything is green. The notification never arrived.

A payment finishes, but its notification scheduler stops doing useful work. The container stays healthy. Clutta watches whether the journey finishes, not just whether the service is alive.

THE JOURNEY YOU WILL WATCH



Same payment ID across all three steps. You approve the expected flow and deadline.

THREE MOMENTS. ONE REAL WORKFLOW.

- 1 Learn what finished looks like** ./lab start
 Generate 600 distinct fake payments through real Node.js services and PostgreSQL. Review the proposed flow and cited runs in Clutta, then approve and activate it yourself.
- 2 Stop delivery. Keep health green.** ./lab break
 The lab verifies 20 fresh healthy payments before pausing the scheduler loop. A new payment misses its delivery deadline. Open the Case File linked in your terminal.
- 3 Repair and verify new work** ./lab repair
 The saved notification queue drains. Open a fresh completed run in Clutta and compare all three steps. The earlier missed deadline remains in history.

What to look for in the Case File

Cited accepted and queued records for the same payment; the expected delivery step and missed deadline. A completion violation is established. An unknown root cause is not invented.

TRY IT IN YOUR OWN ACCOUNT

Sign up for Clutta, then follow the guide to create a lab project, get your workspace key, and open a fresh Codespace. Each step tells you what success looks like.

Start the step-by-step guide →
github.com/sefastech/clutta-playground

Open in Codespaces →

Repeat the fault, fork the lab, or explore another incident with sanitized evidence in [Analyze](#). Analyze is one-time interpretation, not live monitoring or Case File creation. Finish with `./lab stop`.